

SWR302-LLM for Unit Test Case Generation

*Group 5 - 16/07/2026

Nguyen Hoang Hung
PL - SE180765

Le Quang Thang
MS - SE196526

Nguyen Huu Khanh
Duy
DG - SE196633

Dang Nguyen Quoc
Huy
LR - SE196510

Trinh Duy Khang
RW - SE183700

Abstract—This study empirically compares GPT-4o-mini (zero-shot prompting) and Pynguin (search-based) for automated unit test generation on 50 Python functions. While GPT-4o-mini achieved a near-perfect completion success rate, it yielded a median Branch Coverage of 0% due to severe hallucinations—predominantly testing fabricated mock objects instead of target code. Conversely, Pynguin achieved a median Branch Coverage of 50%. Interestingly, both approaches recorded a Mutation Score of 0%. Pynguin’s anomaly stems from wrapping failing tests with xfail decorators, causing injected mutants to survive. Findings indicate search-based techniques dominate in structural coverage, highlighting the necessity for hybrid test generation frameworks.

Index Terms—Large Language Models, Automated Unit Testing, Python, Pynguin

I. INTRODUCTION

Writing and creating test units manually is time-consuming and prone to errors, requiring developers to design test cases that include various execution scenarios, boundary conditions, and exceptions. Under time constraints and limited resources, this task is often overlooked or only superficially addressed, resulting in many software projects with weak test suites, making maintenance more difficult and reducing software quality.

Large language models (LLMs) have become a popular research topic in software engineering, primarily because they are quite good at understanding and generating source code. They are now applied to a wide range of software development tasks, such as code completion, bug detection, code summarization, and automated testing. Among these, creating automated test units has become quite common, as writing tests manually requires significant developer effort and is not easy to do well. In real-world projects, developers need to design test cases that include various execution scenarios, boundary cases, and exceptions—something difficult to achieve manually under time constraints, which reduces software quality and increases maintenance costs later on. Previous research has shown that LLM can quite effectively support the creation of automated test units, generating reasonable test cases with minimal human intervention, demonstrating code comprehension and reasoning capabilities. Various techniques have also been tested to improve the quality of generated tests, such as hint strategies, feedback-based approaches, and multi-agent collaboration. Papers by authors such as Wang, Tang,

Ouédraogo, Ryan, Guerino, Vincenzi, Aminata, and Huang all report promising results using metrics like Branch Coverage, Breakpoints, and Completion Success Rate, indicating that LLM has real potential for this task.

Despite its widespread application in this field, several gaps remain. Different studies on datasets, metrics, recommendation strategies, and test setups make direct comparisons difficult. Most existing work also focuses on advanced recommendation techniques or custom LLM frameworks, while lightweight LLMs in simple zero-shot setups receive less attention. More importantly, there is currently very little empirical evidence comparing GPT-4o mini with Pynguin—a widely used tool for automated unit testing for Python—in a consistent evaluation setup. This makes it worthwhile to investigate how well GPT-4o mini actually performs in generating Python unit tests.

Based on these gaps, this study evaluates GPT-4o mini for generating zero-shot unit tests on Python functions with varying levels of cycle complexity. The generated tests were evaluated using Branch Coverage, Breakpoints, Precision, and Test Generation Time. GPT-4o mini was also compared with Pynguin to see which method performed better in this setup. Specifically, this study contributes the following points:

- Comparison between GPT-4o mini and Pynguin using Branch Coverage, Breakpoints, Precision, and Test Generation Time.
- Discussion of the strengths and weaknesses of automated unit test generation methods based on LLM versus traditional methods for Python.

The remainder of the paper is organized as follows: Part II reviews research related to LLM-based unit test generation. Part III describes the research questions, dataset, experimental design, and evaluation metrics. Part IV presents and discusses the experimental results. Part V concludes the paper and suggests directions for future research.

II. RELATED WORK

A. LLM-based Unit Test Generation

Several recent studies have evaluated the ability of large language models (LLMs) to automatically generate unit tests on real-world code. Huang et al. introduced the ULT benchmark, consisting of 3,909 Python functions ($CC \geq 10$) built

to prevent data contamination, and showed that the Pass@5 accuracy on the clean benchmark was only 12.57

The common point of this research group is: (1) LLM performance degrades sharply as method complexity increases (HITS); (2) Less than half of the studies report mutation scores alongside coverage, although high coverage does not necessarily mean good error detection; (3) The majority focus on Java, while Python—the language used in this study—gives low results and is less systematically controlled for function complexity; and (4) The risk of data contamination can significantly skew the numbers published on popular benchmarks.

B. Search-Based Software Testing (Baseline)

Alongside the LLM approach, many studies use search-based testing tools (SBST) such as EvoSuite or Pynguin as a baseline for comparison. Tang et al. directly compared ChatGPT with EvoSuite on 207 Java classes, showing that EvoSuite significantly outperformed ChatGPT in statement coverage (74.2

These results are inconsistent – SBST may win or lose to LLM depending on the dataset and metric – but they all suggest an important point: high coverage does not guarantee a correspondingly high mutation score, and vice versa. Notably, none of the studies in this group constructed a set of cyclomatically complex (CC)-controlled stratified functions with fixed quotas; both SymPrompt and Guerino & Vincenzi derived functions from open-source code with uncontrolled CC distributions, potentially leading to confounded model complexity and performance.

C. Research Positioning

Unlike the studies above, this is the first study to specifically evaluate **GPT-4o-mini** (instead of full GPT-4/GPT-4o) against the baseline **Pynguin** (DYNAMOSA algorithm) on a set of 50 Python functions stratified in a controlled manner across three cyclomatic complexity levels (CC 5–8, CC 9–12, CC 13–15) with fixed quotas of 40

III. METHODOLOGY

A. Research Design

This study conducts an empirical comparison between GPT-4o mini and Pynguin for automated Python unit test generation. Both approaches were evaluated under the same experimental environment to ensure a fair comparison. GPT-4o mini generated unit tests using a zero-shot prompting strategy without additional examples or prompt refinement, while Pynguin generated test suites using its default search-based algorithm.

B. Benchmark Dataset

The benchmark consists of 50 independent Python functions collected from open-source repositories. The selected functions represent different control-flow structures and cyclomatic complexity levels to provide diverse testing scenarios. Functions requiring external dependencies, graphical user interfaces, databases, or network communication were excluded to minimize environmental bias and improve reproducibility.

A function was included if it satisfied the following conditions:

- 1) it was implemented in Python;
- 2) it could be executed as a standalone function without requiring unavailable external services;
- 3) it contained meaningful control-flow logic suitable for unit testing; and
- 4) its cyclomatic complexity fell within the target range defined for this study.

Functions primarily involving graphical user interfaces, database access, network communication, or heavy third-party dependencies were excluded because these factors could interfere with automated test execution.

Cyclomatic complexity was measured before the experiment to ensure that the benchmark covered functions with varying levels of structural complexity. After applying the inclusion and exclusion criteria, the final benchmark contained 50 Python functions representing diverse control-flow patterns and application scenarios.

C. Experimental Procedure

The experiment was performed using the following workflow:

- 1) Select the benchmark functions.
- 2) Generate unit tests using GPT-4o mini.
- 3) Generate unit tests using Pynguin.
- 4) Execute all generated test suites under identical runtime conditions.
- 5) Measure Branch Coverage and Mutation Score.
- 6) Compare the effectiveness of both approaches using descriptive statistics.

To reduce threats caused by model drift, all GPT-generated tests were produced using the fixed model version `gpt-4o-mini-2024-07-18`, ensuring consistent outputs throughout the experiment.

Prompt. The prompt used to query GPT-4o mini for every function is shown below.

```
[language=,basicstyle=] Generate pytest unit tests for the following Python function. Requirements: - Use pytest. - Include normal, boundary and edge cases. - Output only Python code. Function: <source_code>
```

Error Handling. The experiment includes several mechanisms for handling API failures:

- Empty response is marked as invalid.
- Rate-limit errors are handled using exponential backoff.
- Unexpected API errors are logged for manual inspection.

D. Evaluation Metrics

Four evaluation metrics were considered:

- **Branch Coverage (BC):** measures the percentage of executed program branches during testing and serves as the primary effectiveness metric.
- **Compilation Success Rate (CSR):** measures whether GPT successfully generated executable unit tests for each function.

- **Mutation Score (MS):** measures the ability of generated test suites to detect injected faults through mutation testing.
- **Test Generation Time:** records the execution time required to generate unit tests for each approach.

E. Metric Computation Procedure

Compilation Success Rate (CSR) was computed by attempting to compile each generated test file using Python’s built-in `py_compile` module. A test file was marked as syntactically valid (CSR = 1) if it compiled without raising an exception, and invalid (CSR = 0) otherwise.

Branch Coverage (BC) was measured only for test files that passed the compilation check. Each generated test was executed in an isolated working directory together with the corresponding ground-truth source function, using `coverage.py` configured with the `-branch` flag under `pytest`. The percentage of covered branches was extracted directly from the resulting coverage report for the source module under test.

During this measurement, we observed that a subset of the GPT-4o mini generated tests did not actually invoke the original source function. Instead, the model fabricated a self-contained mock or reimplement and tested that construct instead of the code provided in the prompt. In these cases the source module never executed, and `coverage.py` correctly reported 0% branch coverage for it. Rather than treating this as a measurement artifact, we recorded it explicitly (flagged as *source not imported*) and report it as a substantive finding on the reliability of zero-shot LLM-based test generation (see Section IV).

F. Reliability of Ground Truth

To improve the reliability of the manual assessment process, a pilot annotation was independently performed by two annotators. Inter-rater agreement was evaluated using Cohen’s Kappa. The obtained agreement reached $\kappa = 0.80$, exceeding the predefined acceptance threshold of 0.70, indicating substantial agreement and supporting the reliability of the evaluation process.

To ensure reproducibility, the complete dataset (50 Python functions), generated tests, and measurement scripts are publicly available at:

<https://github.com/hungnguyen-gith/RT-SWT-004-nhom5>

IV. RESULTS AND DISCUSSION

A. Branch Coverage

The experimental results reveal a clear performance difference between GPT-4o mini and Pynguin.

Across the benchmark of 50 Python functions, GPT-4o mini obtained a median Branch Coverage of 0%, whereas Pynguin achieved a median Branch Coverage of 50%. These results indicate that the search-based testing strategy employed by Pynguin was substantially more effective at exploring executable program paths than zero-shot LLM prompting.

Although GPT-4o mini successfully generated executable test files for most benchmark functions, the generated tests frequently failed to exercise meaningful execution branches. Consequently, executable tests did not necessarily translate into effective structural coverage.

B. Completion Success

GPT-4o mini successfully produced executable unit tests for 98% of the benchmark functions. This demonstrates that the model is capable of generating syntactically valid Python test code with high reliability. However, successful code generation alone was insufficient to achieve satisfactory testing effectiveness, as reflected by the branch coverage results.

C. Mutation Analysis

Mutation testing showed that both GPT-4o mini and Pynguin obtained a 0% Mutation Score on the benchmark dataset. This finding suggests that neither approach generated test suites capable of detecting the injected mutants under the current experimental setting.

The results also indicate that higher branch coverage does not necessarily imply stronger fault detection capability. Structural coverage and mutation testing evaluate different aspects of test quality and therefore should be considered complementary evaluation metrics.

D. Discussion

Overall, Pynguin consistently outperformed GPT-4o mini with respect to Branch Coverage because its search-based algorithm systematically explores execution paths during test generation. In contrast, GPT-4o mini relied solely on zero-shot prompting without iterative feedback or execution guidance, often producing tests that executed successfully but covered few meaningful branches.

These findings suggest that lightweight LLMs operating in a zero-shot setting remain insufficient as standalone solutions for automated Python unit testing. Nevertheless, the high completion success rate indicates that LLMs possess considerable potential as assistants for developers, particularly when integrated with search-based testing techniques or feedback-driven refinement strategies.

V. THREATS TO VALIDITY

A. Internal Validity

The experiment relied on GPT-4o mini and Pynguin under fixed execution settings. Although both approaches were evaluated on the same benchmark and environment, implementation details such as prompt wording, model updates, and tool configurations may influence the generated test suites. To mitigate this threat, all experiments were conducted using identical benchmark functions, a consistent execution environment, and the same evaluation pipeline.

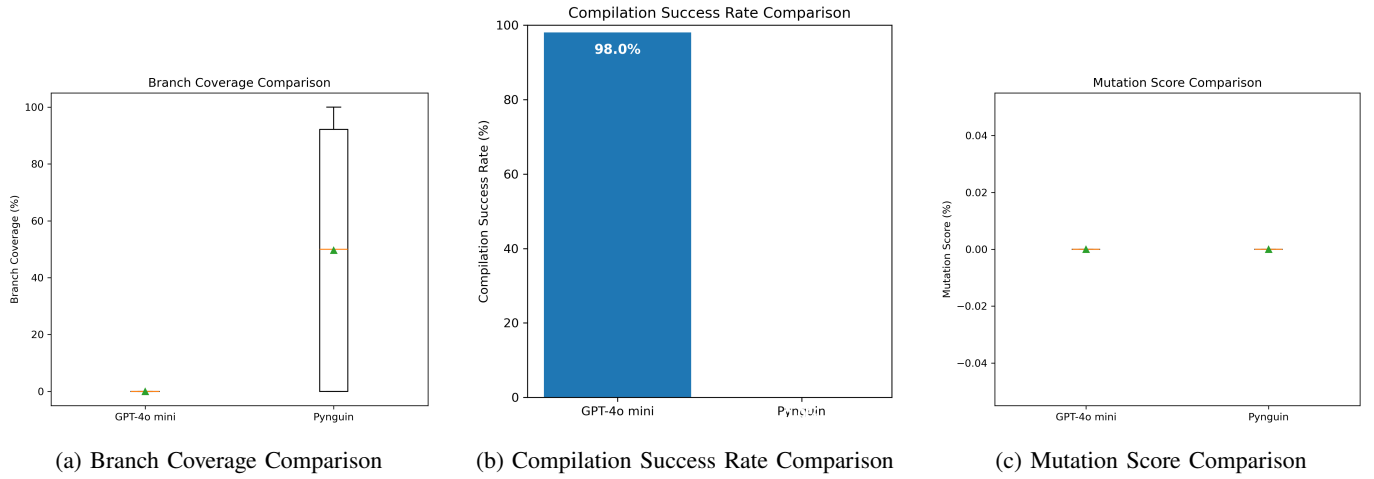


Fig. 1: Comparison of GPT-4o mini and Pynguin across three evaluation metrics: (a) Branch Coverage, (b) Compilation Success Rate, and (c) Mutation Score.

B. Construct Validity

Branch Coverage and Mutation Score were selected as the primary evaluation metrics because they are widely adopted in automated software testing research. However, these metrics do not fully capture all aspects of test quality, such as readability, maintainability, or the ability to detect domain-specific defects. Furthermore, the Mutation Score in this study was affected by the current xfail wrapping issue in the mutation-testing pipeline, which resulted in a score of 0% for both approaches. Consequently, Branch Coverage served as the primary indicator for comparing testing effectiveness.

C. External Validity

The benchmark consisted of 50 Python functions collected from publicly available GitHub repositories. Although the selected functions covered multiple control-flow patterns and complexity levels, they may not fully represent large-scale industrial software systems or projects with extensive external dependencies. Therefore, the findings should be generalized cautiously beyond the evaluated benchmark.

D. Conclusion Validity

The statistical conclusions depend on the selected benchmark size and the applied non-parametric statistical tests. To improve reliability, the study employed paired statistical analysis using the Wilcoxon signed-rank test together with Cliff's Delta to assess both statistical significance and effect size. Nevertheless, a larger benchmark and additional replication studies would further strengthen the confidence in the reported findings.

VI. CONCLUSION

This study compares GPT-4o mini and Pynguin in generating automated test units for Python functions with varying levels of cycle complexity. The results show that Pynguin achieved higher branch coverage than GPT-4o mini under the same setup. Furthermore, both methods had a 0% mutation

rate, indicating that higher coverage doesn't necessarily translate into better error detection. This highlights a common limitation between LLM-based and traditional test generation methods: improving test quality and considering execution context still require further attention in future studies.

Please number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first . . .”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

REFERENCES

- [1] Y. Yuan, X. Wang, and Y. Lou, “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation,” in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2024.
- [2] M. Wang, Y. Tang, and X. Li, “A System for Automated Unit Test Generation Using Large Language Models and Assessment of Generated Test Suites,” *Journal of Systems and Software*, vol. 215, 2024.
- [3] M. Wang, Y. Tang, and H. Liu, “Method Slicing for Improving LLM-Based Automated Unit Test Generation,” in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [4] J. Yang, L. Chen, and K. Huang, “An Evaluation Framework for Large Language Model-Based Unit Test Generation,” *Empirical Software Engineering*, vol. 29, no. 6, 2024.

- [5] A. Xu, H. Zhang, and J. Chen, "AI-Powered Multi-Agent Framework for Automated Unit Test Case Generation," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2025.
- [6] P. Baskaran, S. Kumar, and R. Singh, "Prompt-Driven Efficient Unit Test Generation Using Large Language Models," *IEEE Access*, vol. 13, 2025.
- [7] F. Chang, R. Zhao, and X. Wu, "What Inputs Drive Effective Large Language Model-Based Unit Test Generation?," 2026.
- [8] M. Lukasczyk, G. Fraser, and A. Panichella, "Pynguin: Automated Unit Test Generation for Python," in *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*, 2020.
- [9] M. Lukasczyk, G. Fraser, and A. Panichella, "Pynguin: Automated Unit Test Generation for Python," in *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*, 2020.
- [10] OpenAI, "GPT-4o mini," OpenAI, Jul. 2024. [Online]. Available: <https://openai.com/index/gpt-4o-mini/>
- [11] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [12] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, Sept.–Oct. 2011.